CLI (Command Line Interface)

For the **CLI**, you can provide a Python script that renames a file via command-line arguments. Here's how you could implement it:

Python Code

```
import os
import sys

def  rename_file(old_name, new_name):
   try:
      os.rename(old_name, new_name)
      print(f"File renamed from {old_name} to {new_name}")
   except FileNotFoundError:
      print(f"Error: {old_name} not found.")
   except Exception as e:
      print(f"An error occurred: {e}")

if __name__ == "__main__":
   if len(sys.argv) != 3:
      print("Usage: python rename_file_cli.py <old_filename> <new_filename>")
   else:
      rename_file(sys.argv[1], sys.argv[2])
```

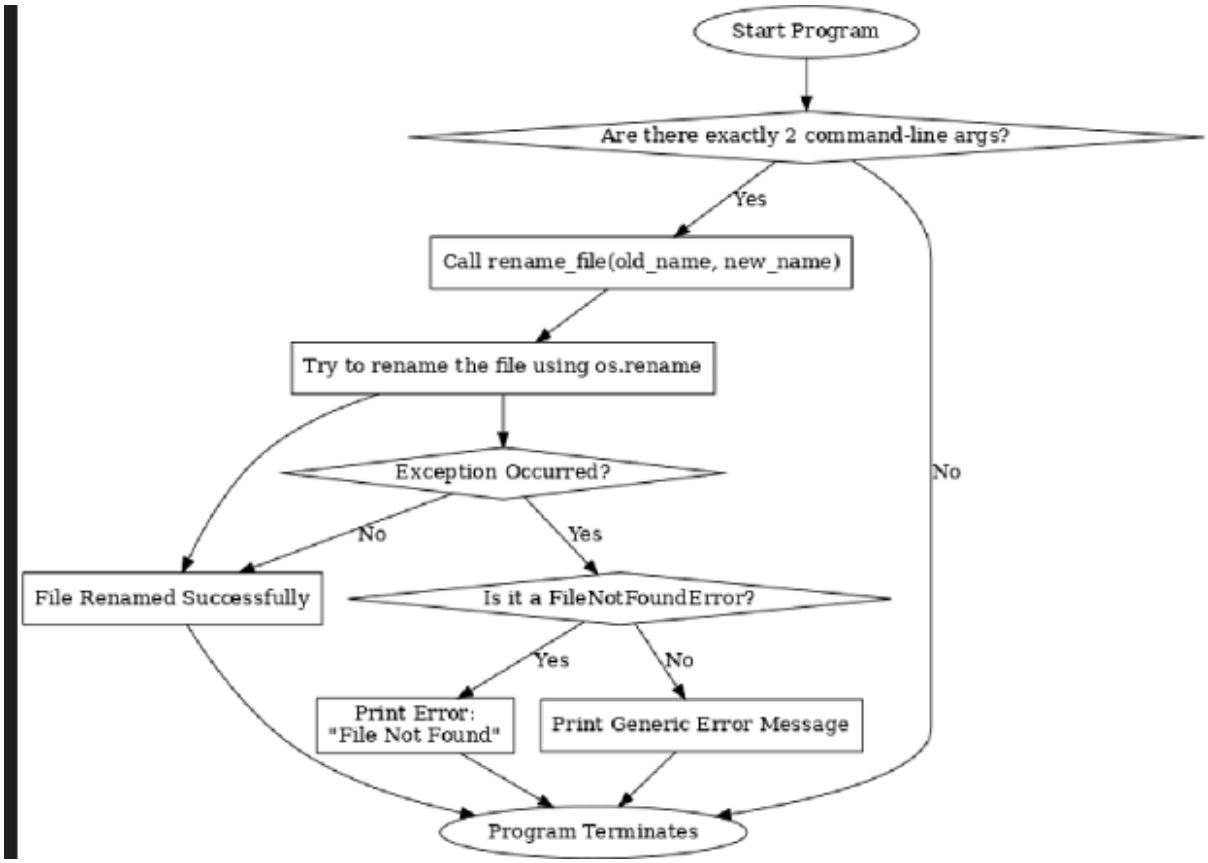
How it works:
The script accepts command-line arguments for the old and new filenames and renames the file.

How It Works:

1. **Imports:**
   - `os` for interacting with the file system.
   - `sys` for accessing command-line arguments.
2. **Function `rename_file`:**
   - Takes two arguments: `old_name` (current filename) and `new_name` (desired new filename).
   - Tries to rename the file using `os.rename()`.
   - Handles `FileNotFoundError` if the original file doesn't exist.
   - Catches any other unexpected exceptions.

To use it:

```
python rename_file_cli.py oldfilename.txt newfilename.txt
```



GUI (Graphical User Interface)

In the **GUI**, we will use **Tkinter** to create a simple interface where the user can type the old and new filenames.

```
import tkinter as tk
from tkinter import messagebox
import os

def rename_file():
    old_name = old_filename_entry.get()
    new_name = new_filename_entry.get()

    try:
        os.rename(old_name, new_name)
        messagebox.showinfo("Success", f"File renamed from {old_name} to {new_name}")
    except FileNotFoundError:
        messagebox.showerror("Error", f"File {old_name} not found.")
    except Exception as e:
        messagebox.showerror("Error", f"An error occurred: {e}")

# Set up the main window
root = tk.Tk()
root.title("File Renamer")
```

```
# Create and place labels, entries, and buttons
tk.Label(root, text="Old Filename:").grid(row=0, column=0)
tk.Label(root, text="New Filename:").grid(row=1, column=0)

old_filename_entry = tk.Entry(root)
old_filename_entry.grid(row=0, column=1)

new_filename_entry = tk.Entry(root)
new_filename_entry.grid(row=1, column=1)

rename_button = tk.Button(root, text="Rename File", command=rename_file)
rename_button.grid(row=2, columnspan=2)

# Start the Tkinter event loop
root.mainloop()
```

How it works:
The GUI allows the user to input the old and new filenames, then click a "Rename File" button. Upon clicking the button, it attempts to rename the file and provides feedback via a message box.

Visual Layout:

- **Window Title:** *File Renamer*
- **Size:** 400x150 pixels (fixed, not resizable)

▯ Old Filename:

(Entry field aligned to the right with enough space to type file names)

▯ New Filename:

(Another entry field right below the first one)

✅ Rename File Button:

- **Color:** Green background with white text
- **Text:** *"Rename File"*
- **Centered below the input fields**

When you click the **Rename File** button:

- A pop-up will appear:
  - **Success Message:** Green tick icon with a confirmation.
  - **Error Message:** Red cross icon if there's an issue.
  - **Warning:** Yellow triangle if inputs are empty or overwriting a file.

File Renamer
File Renamer
Old Filename:
New Filename:
Old Filename
New Filename
Old
New
Rename File
Rename File
Rename File

File Renamer
Old Filename:
New Filename:
Old Filename:
New Filename
Rename File
Rename File

VUI (Voice User Interface)

For the **VUI**, we can use the **SpeechRecognition** library to listen for voice commands. The user will say something like, "Rename oldfilename.txt to newfilename.txt."

```
import speech_recognition as sr
import os

def rename_file_from_voice_command(command):
    # Extracting old and new filename from the command
    try:
        words = command.split(" ")
        old_name = words[1]
        new_name = words[3]

        os.rename(old_name, new_name)
        print(f"File renamed from {old_name} to {new_name}")
    except Exception as e:
        print(f"Error: {e}")

def listen_for_command():
    recognizer = sr.Recognizer()
    mic = sr.Microphone()

    print("Listening for command to rename a file...")
    with mic as source:
        recognizer.adjust_for_ambient_noise(source)
        audio = recognizer.listen(source)

    try:
        command = recognizer.recognize_google(audio)
        print(f"Command received: {command}")
        rename_file_from_voice_command(command)
    except sr.UnknownValueError:
        print("Sorry, I couldn't understand the command.")
    except sr.RequestError as e:
        print(f"Could not request results from Google Speech Recognition service; {e}")

if __name__ == "__main__":
    listen_for_command()
```

How it works:

- The script listens for voice input using the microphone and recognizes the command using Google's speech recognition engine.
- The user can say something like: "Rename oldfilename.txt to newfilename.txt."
- The script extracts the filenames and attempts to rename the file.

Installation:
Ensure you have the necessary libraries installed:

```
pip install SpeechRecognition
pip install PyAudio  # For microphone input
```

User Satisfaction Comparison

To assess user satisfaction, you can compare the interfaces on the following aspects:

1. **Efficiency** (Time to complete task):
   - **CLI**: Most efficient for advanced users.
   - **GUI**: Moderate efficiency, dependent on visual navigation.
   - **VUI**: Dependent on speech recognition accuracy and environmental conditions.
2. **Learning Curve**:
   - **CLI**: Steep for non-technical users.
   - **GUI**: Easy to learn for most users.
   - **VUI**: Very shallow, as it mimics natural language commands, but voice accuracy issues might hinder this.
3. **Error Handling**:
   - **CLI**: Minimal feedback, error prone for typos.
   - **GUI**: Provides visual feedback with error messages.
   - **VUI**: Dependent on recognition; can misunderstand commands if not clear.

4. **User Satisfaction**:
    - **CLI**: Likely to be rated low by non-technical users, but advanced users will be very satisfied.
    - **GUI**: Likely to be rated highly by most users due to ease of use.
    - **VUI**: Satisfaction depends on voice recognition accuracy; users may find it convenient for hands-free tasks, but errors can cause frustration.

How to Measure User Satisfaction

You can implement a simple survey to measure user satisfaction after each task completion. For example:

```
def survey():
    print("Rate your satisfaction with the following interfaces (1-5):")
    cli_satisfaction = int(input("CLI (1-5): "))
    gui_satisfaction = int(input("GUI (1-5): "))
    vui_satisfaction = int(input("VUI (1-5): "))

    print(f"\nYour satisfaction ratings:")
    print(f"CLI: {cli_satisfaction}")
    print(f"GUI: {gui_satisfaction}")
    print(f"VUI: {vui_satisfaction}")
```

Final Thoughts

- **CLI** is best for power users who need speed and control.
- **GUI** is ideal for most users due to its intuitive nature and ease of use.
- **VUI** provides a hands-free experience, though it may be prone to inaccuracies